Assignment 3: Breaking the Ratchet

Team Size: Individual or Pair
Deadline: 2nd November 2026 (Monday) 11:59 pm

Overview

Modern instant messaging apps, including WhatsApp and Signal, use cryptographic protocols to secure messages. One such protocol is the Double Ratchet Algorithm, developed by Signal. Combined with an initial key agreement such as X3DH, it provides end-to-end encryption (E2EE) with strong security properties, including forward secrecy and post-compromise security.

In Assignment 1, you implemented a protocol closely modeled on TLS to learn the internal mechanisms of a secure channel. This assignment is closer to how real-world instant messaging apps are built: rather than securing the connection between a client and a server, the protocol protects messages end to end, so even the server relaying them cannot read them, provided Alice and Bob have authentic copies of each other's identity keys.

You will investigate a flawed implementation of a Double Ratchet-based messaging system and examine how implementation errors in message processing, session establishment, and trust management can undermine these guarantees.

Learning Objectives

1. Explain the state transitions of the Double Ratchet algorithm.
2. Analyze a messaging protocol implementation using packet captures and runtime logs.
3. Identify vulnerabilities in message processing and initial key establishment.
4. Develop proof-of-concept attacks in C/C++ against a controlled lab environment.
5. Measure the availability impact of protocol-level vulnerabilities.
6. Propose and justify secure implementation changes using pseudocode.

Environment & Setup

Tools

- Docker with the Docker Compose plugin

- Wireshark, tcpdump, or tshark for network traffic analysis
- C/C++ for attack implementations
- Process or container monitoring tools for resource measurements
- Python 3 for optional analysis and plotting

Pull the Docker Images

You have been provided with a Docker Compose configuration. Pull the required images using:

```
docker compose pull
```

Start the Lab

Run the system in three terminals (the Compose file sets a shared network named `as3net`).

```
# Terminal 1 — server (clients reach it as `as3_server`)
docker compose up as3_server

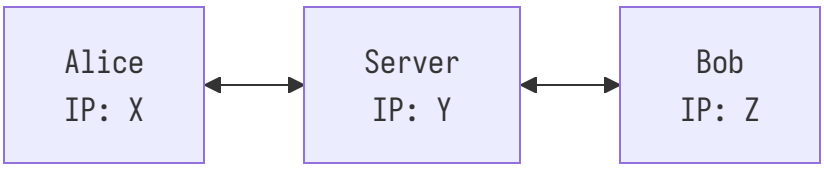
# Terminal 2 — Bob (register + publish a bundle FIRST, per Phase 0)
docker compose run --rm bob

# Terminal 3 — Alice (sends first)
docker compose run --rm alice
```

On client you can manually specifiy the server ip address and port using `--serve-addr` and `--port` flags.

```
docker compose run --rm bob --serve-addr <IP> --port <PORT>
```

The Compose configuration creates a shared network named `as3net`.



Client Interface

The client provides the following menu:

- **Select user to message:** Enter the receiver's username to establish a session and enter the chat loop.
- **Upload prekey bundle:** Replenish the client's one-time prekeys, if supported by the implementation.
- **Exit:** Terminate the client.

Inside the chat loop, type a line to send a message. Press `Ctrl+D` to leave the chat and return to the menu.

Each client prints protocol logs during communication. Use these logs alongside packet captures when investigating the implementation.

Resetting

Use `docker compose down`. It stops the server and clears its in-memory state. Before running experiments, inspect the configuration and determine how to reset the server state, client state.

Note: Each client's data is saved in a JSON file. To clear the data, use:

```
# List all volumes
docker volume ls

# Delete the volume (e.g. student_as3state)
docker volume rm <volume_name>
```

Understanding the Double Ratchet Algorithm

Before bringing up the containers, make sure you can explain how the Double Ratchet Algorithm works. It gives both parties a fresh key for every message, so that compromising one key reveals neither past messages (**forward secrecy**) nor future ones (**post-compromise security**/future secrecy). Read **Section 2 (Overview)** of the Double Ratchet specification first. It combines:

- **The DH ratchet:** Each party holds a current Diffie-Hellman key pair and sends its current public key in every message. When a party receives a *new* public key, it performs a DH, mixes the result into the root chain, and generates a fresh key pair for its next send.
- **The root chain:** A one-way KDF chain; each DH output produces a new root key plus a fresh chain key.
- **The sending and receiving chains:** Symmetric KDF chains that advance once per message to produce the message key that encrypts/decrypts the payload. Your

sending chain is the peer's receiving chain.

Note: **A DH ratchet step happens exactly when the sending side flips. In our version whenever the sending side changes a new DH ratchet step is triggered.**

Note: To understand how the initial root keys are generated, read X3DH specification.

State Diagram Task

Task: Draw a **hand-drawn state diagram** showing the state transitions of Alice and Bob for the message sequence below.

Assume Bob has already uploaded its pre-key bundle to the server and Alice wants to communicate with Bob.

Required Message Sequence

1. Alice sends message `A1` to Bob.
2. Alice sends message `A2` to Bob using the same DH ratchet key as `A1`.
3. Bob replies with message `B1` using his new DH ratchet key.
4. Bob sends message `B2` using the same DH ratchet key as `B1`.
5. Alice sends message `A3` using her new DH ratchet key.
6. Bob sends message `B3` using the new key generated by his next DH ratchet.

Your Diagram Must Show

For each message:

1. The DH key pair held by each party, including public-key changes.
2. The root key (`RK`) before and after each relevant DH ratchet.
3. The sending and receiving chain keys (`CKs` and `CKr`).
4. The message key (`MK`) derived for the message.
5. Whether the transition performs a DH ratchet or only advances a symmetric chain.
6. Which sending chain corresponds to which receiving chain on the peer.

Use symbolic identifiers such as `RK0`, `RK1`, `CK_A0`, and `MK_A1`. You do not need to calculate actual cryptographic values.

Follow the diagram style in Section 2 of the Double Ratchet specification. You may add intermediate states to make the transitions clear.

Submission: Draw the state diagram by hand, then photograph or scan it and save the file as `state_diagram`. Bring the original hand-drawn diagram with you for the viva.

Breaking the Double Ratchet Algorithm

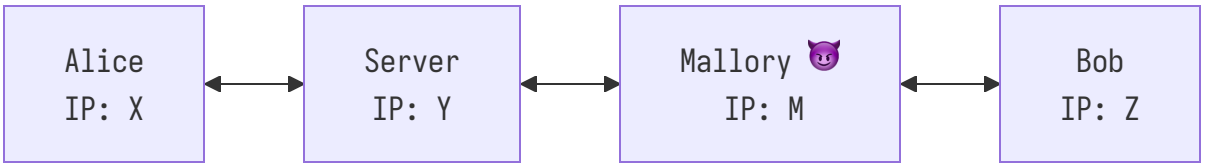
After completing the state diagram, start the server and clients. Analyze the implementation using packet captures and protocol logs.

Note: Before each attack clear the client state.

Attack 0

Create a MITM attacker who is sitting between the server and Bob. When any client sends a message to Bob, what happens if the attacker modifies the header so that the message number is far ahead of the current one? What does the receiving client do? How far can you push it?

Scenario



Note: In the above diagram Mallory is the attacker.

Tasks

A. Analyze message processing

1. Identify the message number and previous message number field and determine how it is represented in the protocol.
2. Determine whether modifying the field invalidates message authentication or whether the attacker can influence the receiver's processing through the supplied implementation.

B. Implement the attack

Write a C/C++ proof of concept that performs the required message-counter manipulation against the lab environment. Document the assumptions that make the manipulation possible. If the header is authenticated and cannot be modified successfully, explain that result and investigate whether another implementation flaw permits the intended resource-exhaustion behavior.

C. Measure the impact

Plot Bob's CPU usage and memory consumption as the counter gap increases (use `docker stats`).

D. Propose a fix

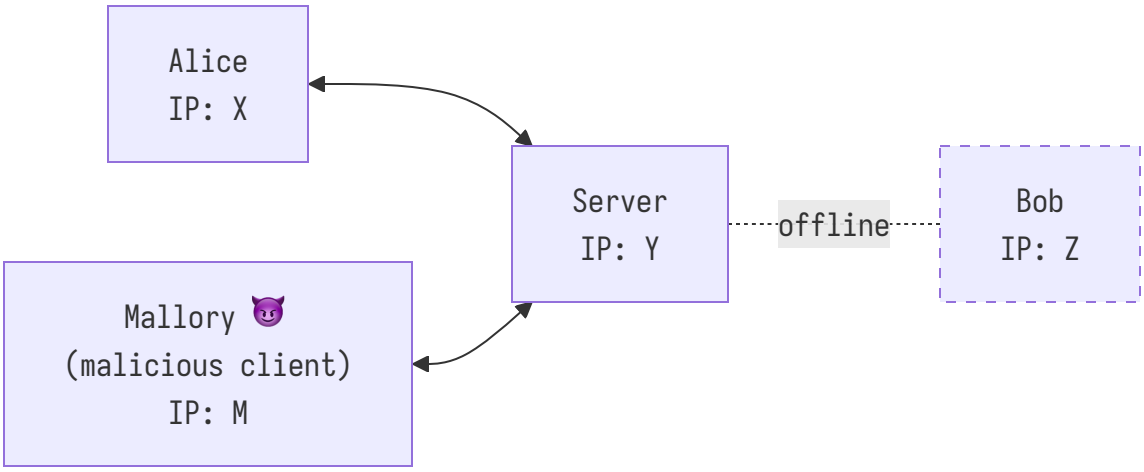
Provide Python-style pseudocode that demonstrates how to fix the implementation flaw.

Attack 1: Prekey Pogo Attack

Bob publishes a prekey bundle that other clients, such as Alice, use to establish an initial secure session. Investigate what happens when Bob's available one-time prekeys (OPKs) are exhausted.

For this assignment, distinguish one-time prekeys from identity keys and signed prekeys. The X3DH specification describes their different roles in initial key establishment.

Scenario



Note: Bob has uploaded his prekey bundle to the server and gone offline. Alice and Mallory are both connected to the server. Mallory is an ordinary client with no special network position.

Tasks

A. Analyze the prekey lifecycle

1. How prekeys are generated, stored, published, and replenished.
2. Whether fetching a bundle consumes an OPK or whether consumption occurs during another stage of session establishment.
3. Whether the same OPK can be allocated to multiple concurrent requests.
4. What happens when no OPKs remain.

B. Implement the attack

Write a malicious C/C++ client that attempts to exhaust Bob's one-time prekey inventory using the behavior exposed by the lab.

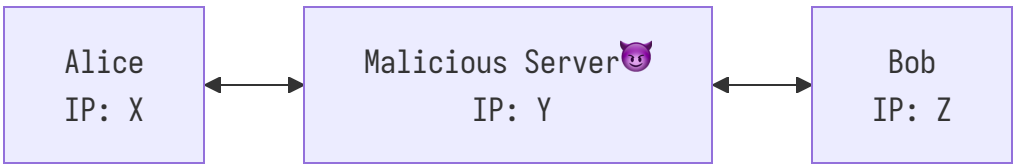
C. Propose a fix

Provide Python-style pseudocode addressing the relevant root cause.

Attack 2: A Malicious Server and MITM

Assume that the messaging server is malicious and attempts to perform a MITM attack between Alice and Bob. This malicious server can relay, delay, drop, replay, or replace messages and key-establishment material to the extent permitted by the implementation.

However, a malicious server cannot automatically decrypt correctly authenticated end-to-end ciphertext. Your task is to identify the specific trust or key-establishment weakness, that enables the server to establish separate sessions with Alice and Bob and read their messages.



A. Analyze the trust model

Investigate:

1. How Alice obtains Bob's identity key and prekey bundle.
2. How identity keys are authenticated or verified.
3. Whether the server can replace key material without detection.
4. Whether Alice and Bob have any independent mechanism to verify each other's identity.

B. Implement the attack

Write a malicious C/C++ server or server-side component that demonstrates the MITM behavior supported by the supplied implementation. For protocol and implementation details refer to `protocol.md`

If the implementation permits key substitution, demonstrate whether the attacker can establish separate sessions with Alice and Bob, decrypt messages received in each session, and forward messages to the other party.

Your evidence must show the distinction between:

- A message that the server merely forwards.
- A message whose plaintext the server can actually recover.
- A message that fails authentication or cannot be decrypted.

If the implementation correctly authenticates all relevant key-establishment material and does not permit the intended MITM, report that outcome and explain which security property prevents the attack. Do not assume success merely because the server is under your control.

C. Propose a fix

Provide Python-style pseudocode showing how to prevent the demonstrated attack, depending on the root cause. Explain the assumptions of your fix and how Alice and Bob can detect a malicious server[1].

Deliverables

1. Photograph or scan of the hand-drawn state diagram, saved as `state_diagram`.
2. Source code for the attacks in a directory named `attacks`, along with a `README.md` describing build and execution instructions.
3. Technical report explaining all flaws/vulnerabilities in the implementation and providing pseudocode (preferably Python 3-style pseudocode), named `report_rollno1_rollno2.pdf`.
4. **Generative AI:** If you used it, add an appendix listing your prompts and how you verified the output.

Submission Requirements

Place all deliverables inside a single directory named:

`as3_rollno1_rollno2/`

Your directory should have the following structure:

```
as3_rollno1_rollno2/
├── state_diagram
├── attacks/
│   ├── README.md
│   ├── attack0.cpp
│   ├── attack1.cpp
│   └── attack2.cpp
└── report_rollno1_rollno2.pdf
```

Compress the directory into `as3_rollno1_rollno2.tar.gz` and submit it.

```
tar -czvf as3_rollno1_rollno2.tar.gz as3_rollno1_rollno2/
```

References

- The Double Ratchet Algorithm
- The X3DH Key Agreement Protocol
- Double Ratchet Messaging Encryption - Computerphile
- How Signal Instant Messaging Protocol Works (& WhatsApp etc.) - Computerphile

---

1. Interested folks can further read a paper related to this problem (This is not part of assignment). Link to the paper: Automatic Detection of Fake Key Attacks in Secure Messaging ↩